RESEARCH BRIEFING

How will GBIF query and download across DwC-DP datasets?

Identifier collisions, structural flexibility, and the role of gbif/dwc-dp#159

Sources: GBIF & TDWG GitHub issues and code, DwC-DP schemas, TDWG ratification notice, Nodes Implementation Plan 2026, pipelines / datapackages-pipeline repositories

Prepared 25 September 2026 · revised 5 October 2026

1 Reader's guide (jargon, briefly)

This briefing mixes three layers that are easy to confuse. Keep them separate:

Layer	What it is
DwC-DP **package**	What the publisher ships: many tables + declared relationships in a Frictionless zip. Expressive; graph shape varies by dataset.
DwC-DP → DwC-A **projection**	GBIF ingest recipes that **read** selected DwC-DP tables and **write** classic Event/Occurrence verbatim records (+ a few extension types). Hand-authored; not a full-graph walker.
GBIF.org **index**	Elasticsearch documents built from that verbatim surface — largely the same fields and extensions as for DwC-A. Cross-dataset search lives here today.

Table 1: Three layers — publishing format, ingest projection, search index.

Useful terms used below:

- **Structural key (`*_pk` / `*_fk`)** — join columns inside a package; aggregators may remint them when concatenating datasets.
- **Semantic / Darwin Core ID (`eventID`, `occurrenceID`, `agentID`, …)** — publisher meaning and provenance; must survive aggregation unchanged.
- **Governing plan** — which recipe book to use for a dataset: Event-core or Occurrence-core (see Section 7.2).
- **Fragment** — one named join-path recipe ("get accepted identification onto the occurrence").
- **Enrichment** — copy values **onto** an existing core row (still one occurrence).

- **Fan-out** — one core row → many extension rows (e.g. many media links).
- **Prune** — drop recipe branches whose tables are absent from **this** package.

2 Executive summary

Yes — issue #159 is the key design fix for aggregate DwC-DP downloads, but it is only one piece of a larger, still-incomplete GBIF strategy. Early DwC-DP drafts used Darwin Core ID terms as primary keys, so two datasets could not be concatenated without collisions or overwriting publisher IDs. The schemas now separate **structural** package keys (`\*_pk` / `\*_fk`) from **semantic** Darwin Core identifiers (`eventID`, `agentID`, …). Aggregators may remint the former; they must preserve the latter. That makes faithful multi-dataset DwC-DP packages **possible**.

What is **not** yet a finished product roadmap is a native GBIF.org service that lets users filter across the full relational DwC-DP graph and download a multi-dataset DwC-DP. Public evidence points to a **two-track** approach:

1. **Near-term indexing path (under active development):** ingest DwC-DP, run a **hand-authored** projection into GBIF's existing Event/Occurrence verbatim model (limited tables and join paths), and re-use today's search and download stack.
2. **Longer-term package path:** keep DwC-DP as a first-class relational container for publishing and (eventually) aggregate export — now that structural keys can be reminted without overwriting publisher IDs.

The hard part is not only identifiers. Publishers may include any subset of 80 table schemas and omit many relationships. GBIF's response is deliberate: **do not index the full graph**; project a known neighbourhood into DwC-A-shaped records. OBIS can mirror that **projection discipline** with DuckDB: a format-agnostic fragment framework over tables and relationships, fed by thin DwC-A and DwC-DP landers — not two projection stacks, and not a forced uplift of DwC-A into full DwC-DP.

> **Bottom line**
>
> Issue #159 unlocked aggregate DwC-DP **as a download format**. Cross-dataset **query** today is occurrence/event-centric via a **limited, declared** DwC-DP → DwC-A projection in `gbif/pipelines` (not a full-graph indexer). OBIS can copy that discipline with DuckDB + multi-table Parquet downloads, without HDFS.

3 Why this is hard

3.1 From star schema to a flexible graph

Under Darwin Core Archive (DwC-A), almost everything hangs off one **core** file (usually Occurrence or Event). Extensions point back with `coreid`. Aggregators like GBIF and OBIS built search, maps, and downloads around that star: one row ≈ one searchable occurrence (or event), with a bounded set of extension types.

DwC-DP drops the single-core rule. A package is a Frictionless Data Package: a `datapackage.json`, an EML metadata file, and as many CSV/Parquet tables as the publisher needs. The canonical catalogue currently lists on the order of **eighty** table schemas. A marine survey might ship `event`, `occurrence`, `survey`, `survey-target`, and many `*-assertion` tables; a collection dataset might centre on `material`,

provenance, and media; an interactions study might rely on tables that never appear in the first dataset at all.

Publishers also choose, per package, which foreign keys to declare. They may mix:

- **Internal** links that must resolve inside the zip (strong `_fk` → `_pk`), and
- **External** identifiers (ORCIDs, institutional URIs, barcode accessions) that need not resolve inside the package (**weak** foreign keys — see #150).

That is excellent for publishing complex biology without denormalising everything onto occurrences. It is awkward for an aggregator that must:

- Merge many datasets into one download without primary-key collisions,
- Preserve publisher-supplied identifiers that users rely on for provenance,
- Offer a query surface that still works when each dataset's table graph looks different,
- Keep indexing years of DwC-A content that never had DwC-DP intermediary tables (Agent, Protocol, Survey, …).

In short: **publishing** wants maximum expressiveness; **network search** wants a stable, predictable projection. Most of GBIF's visible DwC-DP engineering so far is about bridging that gap — not about exposing the full publisher graph as a query API.

TDWG ratified the Conceptual Model and Data Package Guide on 26 May 2026. The announcement explicitly lists **"development of a GBIF implementation of the Data Package Guide"** as a **next step**, not a completed service. Schema/profile release engineering and IPT/pipeline work are still catching up (#139).

4 Issue #159: the key for aggregate packages

4.1 The collision problem

Tim Robertson opened gbif/dwc-dp#159 (Apr 2026) after noticing that early DwC-DP drafts used Darwin Core ID terms **themselves** as Frictionless primary keys. That works inside a single publisher dataset — identifiers only need to be unique **within the package** — but fails as soon as an aggregator concatenates two packages.

His deliberately simple example: two datasets each have an Occurrence pointing at an Agent via `recordedByID = TimR`. In dataset 1, `TimR` is Tim Robertson; in dataset 2, `TimR` is Tim Richardson. Both also reuse `occurrenceID = 123`. If GBIF builds one DwC-DP download containing both:

- Leaving IDs unchanged breaks uniqueness (`agentID` / `occurrenceID` collide), so Frictionless validation fails and joins become ambiguous.
- Overwriting those fields with globally unique values (or with `datasetKey:` prefixes) **can** restore uniqueness, but then the download no longer faithfully shows what the publisher published in `occurrenceID` / `agentID` — the fields users often need for provenance and round-tripping.

DwC-A already avoided this trap: the archive's structural `id` / `coreid` columns are distinct from mapped Darwin Core terms. GBIF can put a mintable `gbifID` (or crawl-local id) in the structural slot while leaving `occurrenceID` as publisher data. Early DwC-DP had collapsed those two roles into one column. Issue #159 is the community working through how to un-collapse them again for aggregate packages.

4.2 Options debated

The thread (and related #150 on weak foreign keys) explored several families of solutions:

Approach	Idea	Main drawback
Prefix / compound keys	e.g. `datasetKey:localID`, or composite `(datasetKey, entityID)`	Alters every ID field; consumers must parse prefixes; nested aggregates become fragile
Mint GBIF keys into DwC ID fields	Put unique values in `eventID` etc., stash originals elsewhere	Overwrites semantic fields; same problem #159 was trying to avoid
Require publisher GUIDs	Force UUIDs/URIs everywhere	Unrealistic; still not clash-proof across datasets
Separate structural keys	`event_pk` / `event_fk` for joins; keep `eventID` as data	More columns; publishers must learn the distinction

Community consensus (Wieczorek, Robertson, Desmet, Doering, Gros, Shorthouse and others on the thread) converged on separate structural keys. Prefixing was seen as workable as an **implementer policy** that would not require schema changes, but brittle for consumers (substring parsing) and for nested aggregates that already carry prefixes. Requiring publishers to mint true GUIDs was rejected as unrealistic and still not clash-proof. Wieczorek's closing argument framed the decision as premises rather than taste:

- Aggregators **must** be allowed to remint structural keys when building aggregate packages.
- Structural keys **must not** be treated as semantically stable identifiers.
- Darwin Core ID terms **must** remain ordinary data fields that survive aggregation unchanged.

In other words: joins inside a DwC-DP zip use throwaway package keys; meaning and provenance live in `eventID`, `agentID`, and friends.

4.3 What was implemented

Issue #159 was closed (26 Jun 2026) after schemas, QRG, and designer artifacts were updated. The vocabulary now encodes aggregation rules explicitly, for example:

Field	Role	Aggregation rule (schema notes)
`event_pk`	Strong primary key	MAY be changed to guarantee uniqueness
`eventID`	Weak / semantic ID	MUST be preserved
`parentEvent_fk`	Strong foreign key	MAY be changed to preserve the relationship
`recordedByID`	Weak foreign key	May point inside or outside the package

Example shape after the change (same collision scenario as #159). Structural columns glue the zip together; Darwin Core ID columns keep each publisher's original values — even when those values collide across datasets:

```
occurrence.csv
occurrence_pk | recordedBy_fk | occurrenceID | recordedByID
1             | A             | 123          | TimR
2             | B             | 123          | TimR

agent.csv
agent_pk | agentID | preferredAgentName
```

```
A        | TimR    | Tim Robertson
B        | TimR    | Tim Richardson
```

A consumer joins on `recordedBy_fk = agent_pk` for relational integrity, and still reads `occurrenceID` / `agentID` / names for science and provenance. Related design #150 added **weak** foreign keys as informative metadata for fields that sometimes resolve internally and sometimes externally (e.g. an ORCID in `recordedByID` with no matching Agent row). Strong `_fk` fields remain Frictionless-validated when declared in the package.

Answer to "is \#159 the key?"

For **faithful multi-dataset DwC-DP downloads**, yes: without separating structural and semantic keys, GBIF could not offer DwC-DP exports that both validate and preserve publisher IDs. That unblocks the **format**. It does not by itself define how GBIF will **query** flexible graphs or which tables appear in a download.

4.4 Could we just add `dataset_id` to every join?

This is the natural first fix people reach for — and it **is** the right idea for a **lake / warehouse** (OBIS's DuckDB prototype already joins on `(dataset_id, eventID)`). It is **not** a full substitute for #159's structural/semantic split in a **Frictionless DwC-DP download zip**. Why:

Concern	What happens
Where `dataset_id` lives	A DwC-DP **package** is usually **one** dataset. Rows do not carry `dataset_id` unless the aggregator **adds** that column when building a multi-dataset zip. So "just join on dataset_id" is an **aggregator policy**, not something publishers already ship.
Frictionless PK/FK rules	Frictionless validates single-column (or declared composite) keys **inside the package**. Concatenating two packages that both have `occurrence_pk = 1` still fails uniqueness on that column unless you remint keys **or** redefine every primary key as `(dataset_id, occurrence_pk)` and every FK likewise. That is a large, brittle schema change for every table and every consumer.
Semantic fields still collide	Even with composite joins, two rows can share `occurrenceID = 123` and `agentID = TimR`. If those Darwin Core columns are also used as Frictionless PKs (the early-draft mistake), validation and "which TimR?" remain broken. Composite keys on **semantic** IDs also force every tool to treat `(datasetKey, occurrenceID)` as the identity — ugly for citations and round-trips.
What #159 actually buys	Keep `occurrenceID` / `agentID` as **ordinary data** (may collide across datasets). Put uniqueness and joins on remintable `occurrence_pk` / `recordedBy_fk`. Optionally **also** stamp `datasetKey` on every row for lake queries — complementary, not alternative.
OBIS lake practice	In a Parquet lake you often **never** merge two datasets into one Frictionless package. Then `(dataset_id, eventID)` joins are enough for analytics. The pain #159 solves is specifically **one download package that contains many datasets** and must still validate as DwC-DP.

Table 2: Composite `(dataset_id, local_id)` joins vs separate `_pk` / `_fk`.

A. Lake / DuckDB (composite join)

```
occurrence ⋈ event
ON o.dataset_id = e.dataset_id
AND o.eventID = e.eventID
```

Datasets stay in separate partitions or always carry `dataset_id`. Semantic IDs never need reminting because you rarely build one zip.

B. Aggregate DwC-DP zip (#159)

Remint `occurrence_pk`, `event_pk`, …
Remap every `_fk` to match.
Leave `occurrenceID` / `eventID` as published.

Optional: also add `datasetKey` column for humans and lake round-trips — but uniqueness comes from `_pk`, not from parsing semantic IDs.

Figure 1: Two ways to avoid cross-dataset join collisions. Lake analytics can use composite keys; a multi-dataset DwC-DP zip still needs remintable structural keys so semantic IDs stay untouched.

Practical answer

Yes — add `dataset_id` / `datasetKey` to lake joins and to rows in aggregate exports when useful. **No** — that alone does not replace `_pk`/`_fk` for Frictionless multi-dataset packages, and it must not become an excuse to overwrite or composite-key the Darwin Core ID fields users rely on.

5 Beyond keys: the flexibility problem

Even with clean structural vs semantic keys, cross-dataset DwC-DP still raises product questions that #159 does not answer. These are about **which subgraph to materialise**, not about whether IDs can be unique.

Which tables ship in a download?
Suppose a user filters to a taxon across a hundred datasets. Some are Event-core surveys with Humboldt targets and hundreds of event-level assertions; others are occurrence-only citizen-science feeds; others are specimen packages with material derivation chains. A useful download must decide: only matching occurrences? parent events up the hierarchy? every Agent those events reference? shared Protocols? Without a policy, either the zip explodes or users miss context they need.

Optional relationships.
Canonical schemas list many **possible** foreign keys. Each published package declares only the ones it uses. A query planner cannot assume "every occurrence has `organism_fk`" or "every event has a survey." GBIF's projection handles this by **pruning** unused branches (see below). A native multi-table download API would need the same idea: start from a driving set, walk only edges that exist.

DwC-A coexistence.
For years to come, most GBIF content will still arrive as DwC-A. Those archives have no Agent table, no Protocol table, no Survey Target. If aggregate DwC-DP downloads are meant to mix DwC-A-derived and DwC-DP-derived rows, GBIF must **mint** structural keys (and sometimes invent intermediary rows) for the DwC-A majority. Manon Gros stressed this on #159: GBIF already mints `gbifID` today; DwC-DP does not remove that need.

Shared local authorities.
A single institution may publish several thematic datasets that all reuse the same local `agentID` vocabulary from one CMS. An aggregate of **only that publisher's** data can still collide — or, conversely, may need SameAs logic if the aggregator tries to merge agents. Identifier policy cannot assume "one dataset = one ID namespace" is the only hard case.

Validation vs expressiveness.
IPT discussions (ipt#2859) show publishers will omit tables such as Agent and still put ORCIDs in

ID fields. Foreign-key checks must then soften (warn, or treat the link as weak/external) rather than block publication. That is healthy for publishing and another reason aggregators cannot treat every ID-looking field as a hard join.

None of these are solved by `_pk` / `_fk` alone. They explain why GBIF's near-term answer is a **bounded projection** into the occurrence/event world they already operate.

6 What GBIF is actually building (public evidence)

6.1 Standards track (TDWG / DwC-DP)

- Conceptual Model + Data Package Guide ratified 26 May 2026.
- Table schemas / profile still moving through release engineering (#139: move workspace toward TDWG, publish ratified artifacts on `rs.tdwg.org`).
- Feature/implementation report documents publishing use cases (OBIS Broke West Fish, Insektmobilen, etc.) but treats aggregator download architecture as future work.

6.2 Publishing tooling

- IPT v3 workstreams aim to map arbitrary tables (not only star cores) and emit Frictionless `datapackage.json` + CSV + EML (ipt#1736).
- Test IPT instances and example packages (`gbif/dwc-dp-examples`) support publisher onboarding and node training.
- Nodes Implementation Plan 2026 emphasizes capacity, mapping, and mobilization — with isolated mentions of download capability (e.g. Burundi node text), not a finished GBIF.org download format.

6.3 Ingestion / indexing (the practical query path)

Public code shows GBIF leaning on a **project-to-existing-index** strategy, not a generic walk of every DwC-DP FK.

HDFS / pipelines. Work tracked in gbif/pipelines#1365 (ingest DwC-DP to HDFS as Parquet) and landed via PR #1396. Flow:

1. Read package resources (CSV/TSV/Parquet) → write Parquet under a dataset attempt path (`DataPackageConversionPipeline` / `DataPackageConverter`).
2. Convert with `DwcDpVerbatimConverter` into `verbatim.avro` (`ExtendedRecord`) — the same intermediate used for classic DwC-A.
3. Hand off to the existing Event/Occurrence interpretation and indexing stack.

Routing comment in that converter:

> `containsEvents` + event table → Event mapping plan;
> `containsOccurrences` + occurrence table → Occurrence mapping plan;
> otherwise → empty verbatim.

That is the Camtrap DP playbook generalized (camtrap-dp-pipeline): accept a richer package at the edge, then fold it into the occurrence/event stack that already supports cross-dataset search and downloads.

Native package analytics (`gbif/datapackages-pipeline`).
In parallel, Spark jobs can load a Data Package into Elasticsearch indices and ClickHouse tables **per dataset**, named like `schemaName_dp_<datasetKey>`. Primary keys become document IDs; foreign keys

become search aliases so related documents can be found without a global join engine. That is useful for dataset-scoped exploration and for future lakehouse work (Parquet / Iceberg are stated goals). It is **not** yet documented as the way GBIF.org runs cross-dataset occurrence search, and it does not replace the verbatim projection path above.

Validation (`gbif/dwc-dp-analyser`).
A separate DuckDB-backed analyser checks Frictionless structure, DwC-DP profile rules, and PK/FK/type constraints. That improves publishing quality; it does not define aggregation or download policy.

7 How the DwC-DP → DwC-A projection works

7.1 The idea in one picture

GBIF does **not** load the DwC-DP graph into Elasticsearch as-is. For each published package it runs a **fixed recipe book** that:

1. Chooses whether the package is Event-shaped or Occurrence-shaped,
2. Joins only the tables those recipes know about,
3. Writes the familiar DwC-A intermediate (`verbatim.avro` / `ExtendedRecord`),
4. Hands that to the **existing** interpretation and indexing stack.

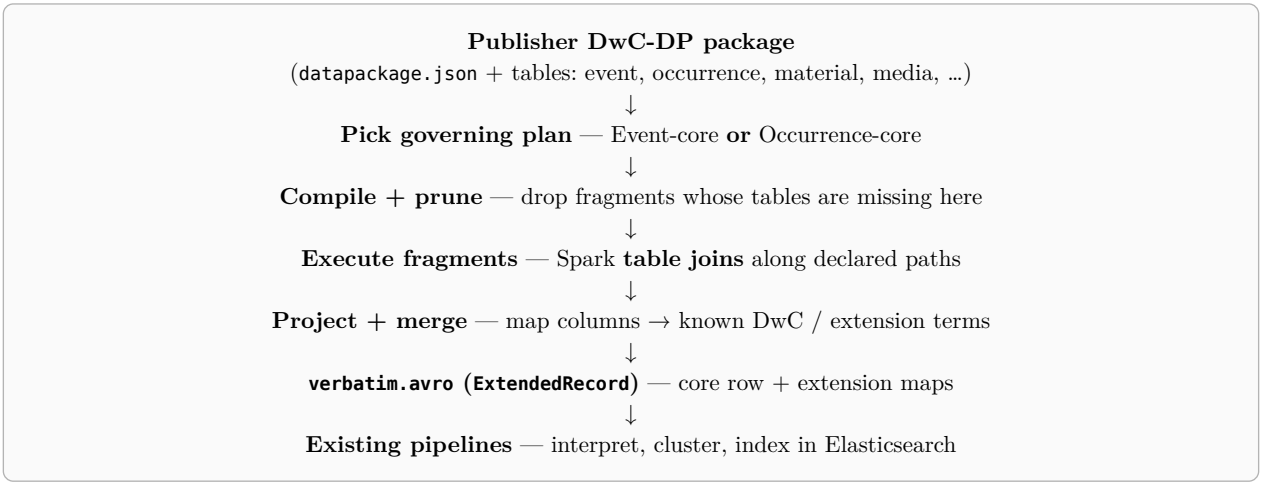


Figure 2: End-to-end ingest: many DwC-DP tables in → one governing plan → Spark joins per fragment → DwC-A-shaped verbatim → same ES index path as classic archives.

The projection is **not** "follow whatever relationships the publisher declared." It is a recipes book: for each kind of target field or extension row GBIF already knows how to index, engineers wrote one or more **fragments** that say which DwC-DP tables to read, which join columns to follow, and how to merge values when several paths compete.

Implementation lives under `…/spark/dwcdp/mapping/`. The package layout matches the lifecycle:

- `definition` — DSL for mapping plans (paths, fragments, aggregations),
- `schema` — official DwC-DP schema graph (what joins are **legal**),
- `compilation` — resolve + prune a plan to one dataset's `datapackage.json`,
- `execution` — Spark DataFrame joins, projection, metrics,
- `config` — GBIF's concrete Event/Occurrence recipes,
- `engine` — facade used by `DwcDpVerbatimConverter`.

7.2 What is a "governing plan"?

A **governing plan** is simply: **which core row type drives the entire conversion for this dataset?**

GBIF's search world still thinks in DwC-A terms: either an **Event** archive (events are the core; occurrences hang under them) or an **Occurrence** archive (occurrences are the core). DwC-DP packages can contain both tables, or only one, or neither. The converter therefore makes a binary choice up front:

Condition	Plan used
Crawl marked event-bearing **and** package has an `event` table	`EventDwcaMapping.current` — one verbatim Event per event row; occurrences become an **extension** under that event
Else if package has an `occurrence` table	`OccurrenceDwcaMapping.current` — one verbatim Occurrence per occurrence row
Otherwise	Empty verbatim set + warning — **no** generic "index whatever tables exist" fallback

Table 3: How `DwcDpVerbatimConverter` picks the governing plan.

That choice matters because the two plans wire **different fragment lists**. An Event-core plan walks parent events, surveys/Humboldt, and nested occurrences; an Occurrence-core plan starts at occurrences and never builds an Event core document. There is no third plan that indexes `organism-interaction` as a first-class searchable unit.

7.3 What is a fragment? How do mappings function?

Think of a **mapping plan** as a cooking menu, and a **fragment** as one recipe card on that menu.

- The **plan** (`EventDwcaMapping.current`) says: "for Event-core packages, run **these** fragments, emit **these** extension types, merge colliding fields with **these** rules."
- A **fragment** is a small, named unit such as `OccurrenceMapping.acceptedIdentification` or `MultimediaMapping.occurrenceMedia`. It declares:
  1. a **path** — start table + ordered joins (`occurrence` → `identification` via `occurrence_fk`, …),
  2. optional **filters** (e.g. `isAcceptedIdentification = true`),
  3. **cardinality** (`exactlyOne` vs `fanOut`),
  4. **targets** — which output fields (or extension rows) to write from which source columns.

Fragments are composed: the Event plan **imports** dozens of them. Engineers reuse the same fragment from both Event-core and Occurrence-core menus where the path is the same. That is why the Java tree looks large — it is a library of recipe cards, not fifty independent indexers.

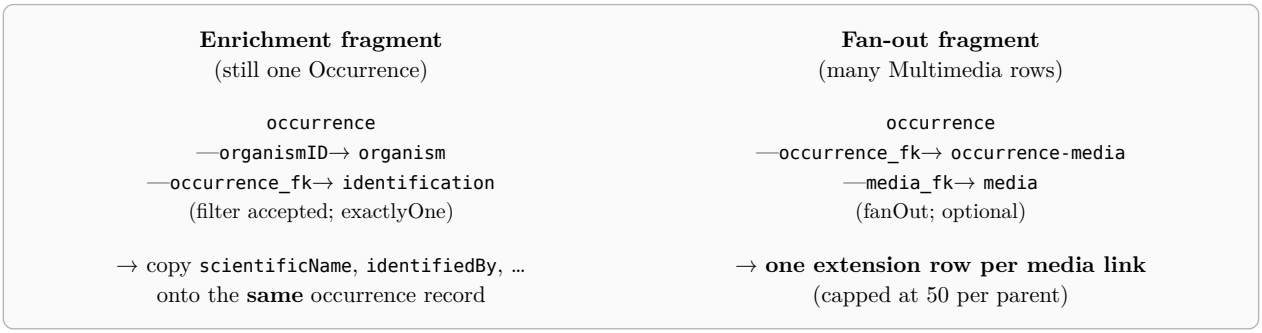


Figure 3: A fragment is a join path + field projection. Enrichment writes onto the core row; fan-out writes many extension rows.

7.4 How does this work technically? (joins, not KV lookups)

Execution is **relational**, not a record-by-record key-value walk of Elasticsearch.

1. Each DwC-DP table in the package is loaded as a Spark `Dataset` (DataFrame) — typically from the Parquet landing written earlier in the crawl.
2. `SparkMappingPathExecutor` starts from the fragment's source table and, for each step in the path, runs a Spark **`join`** to the next table on the named `via` column (e.g. `occurrence.event_fk = event.event_pk`).
3. Filters, optional/required cardinality, and window functions (e.g. count matches, keep row number 1 for `exactlyOne`) are applied on that joined Dataset.
4. Only after the path is materialised are columns renamed/projected into DwC term IRIs and assembled into `ExtendedRecord` Avro.

So: **table joins in Spark** (SQL-shaped), with metrics for how often joins miss or explode. There is no "for each occurrence, GET agent from a KV store" step in this layer. (Elasticsearch comes **later**, after verbatim emission, and stores the **flattened** result — it is not the join engine.)

For OBIS-scale reimplementation the same logic is ordinary DuckDB `JOIN`s / `LEFT JOIN`s with `QUALIFY` / window filters — no need to copy the Java DSL, but do copy the **cardinality and merge rules**.

7.5 Pipeline stages (step by step)

1. **Pick a governing plan** — Event-core or Occurrence-core (see above). No plan ⇒ empty verbatim.
2. **Compile, then prune.**
   `DwcDpMappingEngine` resolves every fragment path against the official schema graph, then a **dataset pruner** deletes branches whose tables or columns are absent from **this** `datapackage.json`. A survey-less package simply never runs Humboldt fragments; missing `media` drops Multimedia branches. Optional structure is handled by **omission**, not by writing a custom plan per dataset.
3. **Execute join fragments.**
   Each remaining fragment becomes a Spark join chain. Cardinality is explicit (next subsection). Optional means "if the join misses, skip this fragment" — not "fail the dataset."
4. **Project and merge fields.**
   Source columns map to Darwin Core or extension terms (`TargetTerms`). When several fragments write the same target (e.g. `identifiedBy` from a direct field **and** from identification-agent roles), merge rules decide the winner: `firstNonNull`, pipe-delimited distinct lists, URN fallbacks such as `urn:gbif:dwcdp:occurrence:` when no semantic ID is present.
5. **Emit verbatim `ExtendedRecord` Avro.**
   One record per Event or Occurrence core row, plus maps of extension row types — the same intermediate classic DwC-A ingest already uses. From there, existing interpretation, clustering, and Elasticsearch indexing proceed largely unchanged.

Core identity prefers the semantic ID, then the structural PK: e.g. try `occurrenceID`, else `occurrence_pk` (and likewise for events). That mirrors the #159 split even inside the projection: the pipeline can mint a stable document key without erasing the publisher ID field on the interpreted record.

7.6 Fan-out, exactly-one, enrichment (cardinality)

These three words are where most confusion sits.

Strategy	Meaning
`optional().exactlyOne()`	Expect **0 or 1** related row after filters. If 0 → skip (no enrichment). If 1 → attach fields. If **>1** → treat as ambiguous and **do not enrich** (accepted identification uses this caution).
`fanOut()`	Expect **0..N** related rows. **Each** match becomes its own extension row (or contributes to a multi-value aggregate). This is how one occurrence yields many Multimedia or eMoF rows. Without fan-out you would either drop all-but-one media or wrongly duplicate the occurrence itself.
Enrichment (onto core)	Write extra **columns** on the existing core record. Row count of the core does not increase.
Extension (new rows)	Write **additional** rows keyed to the core (`coreid` / parent id). Row count of that extension type grows with fan-out.

Table 4: Cardinality strategies in the mapping DSL.

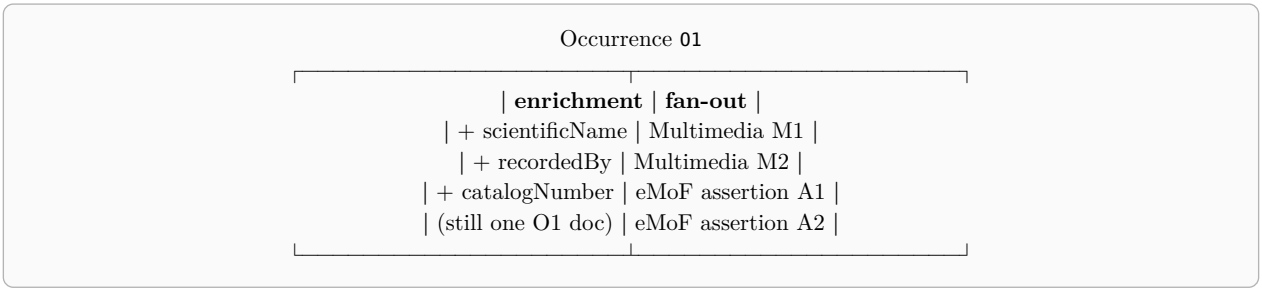


Figure 4: Fan-out vs enrichment. Fan-out multiplies **extension** rows; enrichment widens the **same** core row.

Merge policies when several fragments target the same term.
Common policies: `firstNonNull` (prefer occurrence's own `identifiedBy`, else identification agent, else material collector), `pipeDelimitedDistinct` (protocols, funding attributions), and `firstOrUrnFallback("urn:gbif:dwcdp:…")` when minting a document id.

Weak vs strong joins.
Some edges use structural FKs (`event_fk` → `event_pk`). Others use weak/semantic fields (`evidenceForOccurrenceID` → `occurrence.occurrenceID`, `organismID` → `organism`). The projection must join on the column the schema actually uses — copying GBIF's `via("…")` names — not assume every link is `_fk` → `_pk`.

7.7 A concrete fragment chain (worked example)

Under an Event-core plan, nested occurrences are not "whatever tables exist"; they are built by composing named fragments in `OccurrenceMapping`, for example:

- Start at `occurrence` rows whose `event_fk` points at the parent event.
- Optionally join `event` again for `eventID` identity.
- **Enrich** the occurrence row from `organism`, from the **accepted** `identification` (and at most one unambiguous `identification-taxon`), and from agents/roles on that identification.
- If exactly one `material` row evidences the occurrence (`evidenceForOccurrenceID`), pull catalogue/ownership/geological-context/protocol fields from that material path.
- Separately, **fan out** Multimedia and eMoF-style assertion extensions from occurrence-, material-, survey-, and nucleotide-linked assertion tables.

So "projection" means: **copy and reshape a known neighbourhood of the DwC-DP graph into DwC-A slots**, with documented cardinality rules — not a recursive crawl of every FK until the graph is exhausted.

7.8 Target shape (limited)

Canonical plans only emit this DwC-A-compatible surface — intentionally the extensions GBIF already knows how to interpret and search:

Target row type	Config / notes
Event or Occurrence core	Direct fields + enrichments from related DwC-DP tables
Occurrence (under Event core)	`OccurrenceMapping` nested under Event
Multimedia	`MultimediaMapping` — capped at 50 rows/parent
ExtendedMeasurementOrFact	`AssertionMapping` — assertions → eMoF
Identification	`IdentificationMapping`
Identifier / Reference	`IdentifierMapping`, `ReferenceMapping`
Humboldt	`HumboldtMapping` from survey / survey-target
DNA-derived data	`NucleotideMapping`
ChronometricAge	`ChronometricMapping`

7.9 Source tables and relationships (exact inventory from current mapping)

> **53 sources ≠ 53 index shapes**
>
> The **53** figure is how many DwC-DP **tables the recipes know how to read**. Almost none of them become their own Elasticsearch mapping. They **collapse** into the same surface classic DwC-A already uses: one Event or Occurrence **core** document, plus a short list of **extension row types** (Multimedia, eMoF, Identification, Identifier, Reference, Humboldt, DNA-derived data, ChronometricAge).
> Example: `organism` + `identification` + `identification-taxon` + `identification-agent-role` + `agent` may all be joined, but what lands on the occurrence document is mostly scalar DwC fields (`scientificName`, `identifiedBy`, …). Five assertion tables all fan out into **one** extension type: ExtendedMeasurementOrFact.
> So the index does **not** grow a new type per DwC-DP table — the recipes are a **many-to-few** funnel into today's Event/Occurrence stack.

The report already explained **how** projection works; this section lists **what** `EventDwcaMapping.current` / `OccurrenceDwcaMapping.current` in `gbif/pipelines` (`dev`, Sep 2026) actually touch. Source: string/`via(...)` analysis of `…/mapping/config/`. Counts are against the official DwC-DP table-schema catalogue (77 JSON schemas on `tdwg/rs.tdwg.org`).

Headline: 53 official tables are **readable sources** in the mapping configs; **24** are not. After projection, the **written** shape is still only the target row types in the previous section. Joins are **hard-coded** `via("…")` columns — there is no step that loads `foreignKeys` from a package's `datapackage.json` and walks them all. Dataset pruning only **drops** branches whose tables/columns are missing; it does not **discover** new edges.

7.9.1 How the funnel fits the current index

What happens	Examples
Flatten onto **core** Event/Occurrence fields (enrichment)	`protocol` / `provenance` / `geological-context` / `agent` / accepted `identification` → `samplingProtocol`, `scientificName`, `recordedBy`, catalogue fields, …
Fan out as **Multimedia** rows	`event-media`, `occurrence-media`, `material-media`, `chronometric-age-media`, + `media` / usage-policy / creators — still one extension type, ≤50/parent
Fan out as **eMoF** rows	`*-assertion` from event, occurrence, material, survey, nucleotide-analysis, molecular-protocol, chronometric-age
Fan out as **Identification** / **Identifier** / **Reference**	identification history + DNA IDs; `*-identifier`; `*-reference` / `bibliographic-resource`
Fan out as **Humboldt**	`survey` + `survey-target` (+ protocols) → Humboldt extension on Event
Fan out as **DNA-derived** / **ChronometricAge**	`nucleotide-*` / `molecular-protocol`; `chronometric-age` (+ agents/protocols)
Never becomes its own index type	`material` is not a GBIF.org core — it only **enriches** occurrences or feeds Multimedia/eMoF/DNA/Identifier rows

Table 5: Many DwC-DP sources → few DwC-A / index targets.

After `verbatim.avro` is emitted, **the same** interpretation / clustering / Elasticsearch pipelines run as for DwC-A. That is why "53 tables" still "fits": the search model did not widen; only the **ingest neighbourhood** did.

7.9.2 Existing terms — not a new index vocabulary

Does DwC-DP ingest **add terms to the GBIF.org indexes**? **Essentially no.** The projection's job is to emit **already-known** DwC / Dublin Core / extension term IRIs into `ExtendedRecord`, then hand off to the **same** interpreters that DwC-A already uses.

Evidence in `TargetTerms`:

1. **Resolve to known terms.** Local names are looked up in GBIF's `TermFactory` (`DwcTerm`, `DcTerm`, `EcoTerm`, …). If the name is a known term, the **qualified IRI** is written (e.g. `scientificName` → `http://rs.tdwg.org/dwc/terms/scientificName`).
2. **Explicit renames** map DwC-DP column names onto **existing** DwC fields when labels differ — e.g. `eventConductedBy` → `recordedBy`, `eventConductedByID` → `recordedByID`, `occurrenceReferences` → `associatedReferences`, `accessURI` → `dc:identifier`.
3. **Unknown names are dropped** unless they sit on a small **raw-output allowlist** that is part of an **existing extension contract** (not a new ES mapping invented for DwC-DP): e.g. Identifier-extension keys (`identifierType`, `identifierTypeIRI`, …), `isAcceptedIdentification`, a few nucleotide/DNA extension fields, plus a couple of retained raw keys such as `georeferencedByID` on Event.
4. Humboldt fields use the **existing** Humboldt (`EcoTerm`) namespace where names collide with DwC.

Landing place	What it is
Core Event/Occurrence fields	Standard DwC terms already interpreted/indexed for DwC-A (`eventID`, `scientificName`, `decimalLatitude`, `samplingProtocol`, `recordedBy`, …)
Registered extensions	Same rowTypes as DwC-A: Multimedia, eMoF, Identification, Identifier, Reference, Humboldt, DNA-derived data, ChronometricAge — with their **existing** term sets
Allowlisted extension keys	A few extension-local properties kept verbatim so those extensions still work; not new GBIF.org search facets by themselves
Dropped DwC-DP-only columns	Anything `TermFactory` does not know and that is not allowlisted never reaches verbatim — and therefore never reaches the index

Table 6: Where projected values land — vocabulary reuse, not index invention.

Caveat (timing, not DwC-DP-specific): when TDWG **ratifies** a new DwC term and GBIF **later** adds it to interpreters/ES (e.g. record-level `projectID` / funding terms), DwC-DP recipes can start emitting that term **because it became a known term** — the same path DwC-A would use. That is vocabulary evolution upstream of the projection, not "DwC-DP invented a new index field." The

parallel `datapackages-pipeline` per-table ES indices are a **different** product surface and are not how GBIF.org occurrence/event search is documented to work today.

7.9.3 Tables in the current projection (53)

Neighbourhood	Tables
Cores	`event`, `occurrence`
Taxon / organism path	`organism`, `identification`, `identification-taxon`, `identification-agent-role`, `identification-reference`
Agents / policy	`agent`, `usage-policy`, `media-agent-role`, `material-agent-role`
Context / method	`geological-context`, `protocol`, `protocol-reference`, `provenance`
Material path	`material`, `material-agent-role`, `material-assertion`, `material-geological-context`, `material-identifier`, `material-media`, `material-protocol`, `material-provenance`, `material-reference`, `material-usage-policy`
Survey / Humboldt	`survey`, `survey-target`, `survey-protocol`, `survey-identifier`, `survey-assertion`, `survey-reference` (code also joins a `survey-survey-target` junction in `HumboldtMapping`; that name is **not** in the current official table-schema list — treat as schema-drift / legacy alias to watch)
Event junctions	`event-assertion`, `event-identifier`, `event-media`, `event-protocol`, `event-provenance`, `event-reference`
Occurrence junctions	`occurrence-assertion`, `occurrence-identifier`, `occurrence-media`, `occurrence-reference`
Media	`media`, `media-agent-role`, `media-usage-policy`
Molecular	`nucleotide-analysis`, `nucleotide-analysis-assertion`, `nucleotide-sequence`, `molecular-protocol`, `molecular-protocol-assertion`, `molecular-protocol-reference`
Chronometric	`chronometric-age`, `chronometric-age-assertion`, `chronometric-age-media`, `chronometric-age-reference`
Literature	`bibliographic-resource`

Table 7: DwC-DP tables referenced by GBIF's Event/Occurrence projection recipes.

7.9.4 Relationship columns actually walked (`via`)

These are the join **handles** named in fragment code (structural `_fk` after #159, plus a few weak/semantic IDs):

Kind	Columns
Structural FKs	`event_fk`, `parentEvent_fk`, `occurrence_fk`, `materialEntity_fk`, `media_fk`, `survey_fk`, `surveyTarget_fk`, `protocol_fk`, `eventProtocol_fk`, `occurrenceProtocol_fk`, `georeferenceProtocol_fk`, `samplingProtocol_fk`, `samplingEffortProtocol_fk`, `assertionProtocol_fk`, `provenance_fk`, `geologicalContext_fk`, `identification_fk`, `agent_fk`, `reference_fk`, `usagePolicy_fk`, `chronometricAge_fk`, `nucleotideSequence_fk`, `molecularProtocol_fk`
Weak / semantic	`evidenceForOccurrenceID` (material → occurrence), `organismID` (occurrence → organism), `identifiedByID` / `chronometricAgeDeterminedByID` (→ agent)

Table 8: Join columns used by the projection (not an exhaustive DwC-DP FK catalogue).

7.9.5 Core relationship paths (the ones that matter)

Path	Join sketch
Occurrence → Event	`occurrence --event_fk--> event` (optional, `exactlyOne`)
Event → parent Event	`event --parentEvent_fk--> event` (optional, `exactlyOne`)
Occurrence → Organism	`occurrence --organismID--> organism` (weak; optional, `exactlyOne`)
Accepted identification	`occurrence --occurrence_fk--> identification` filter `isAcceptedIdentification=true` (`exactlyOne`; >1 accepted ⇒ no enrichment)
Identification taxon / agents	`identification --identification_fk--> identification-taxon; … --> identification-agent-role --agent_fk--> agent`
Evidence material	`occurrence <--evidenceForOccurrenceID-- material` (optional, `exactlyOne`)
Material enrichments	`material --materialEntity_fk--> material-{protocol,provenance,geological-context,usage-policy,media,identifier,assertion,reference}` then on to `protocol` / `provenance` / `geological-context` / `usage-policy` / `media` / …
Event protocols / provenance	`event --eventProtocol_fk|georeferenceProtocol_fk--> protocol; event --event_fk--> event-protocol --protocol_fk--> protocol`; same pattern for `event-provenance`
Survey (Humboldt)	`event --event_fk--> survey`; survey protocols/effort; targets via survey↔target link; identifiers/assertions/references fan-out
Multimedia	`*-media --media_fk--> media` (+ `media-usage-policy`, `media-agent-role`); capped at **50** media rows per parent
eMoF assertions	fan-out from `event|occurrence|material|survey|nucleotide-analysis|molecular-protocol|chronometric-age-*-assertion`
DNA-derived data	`material|event → nucleotide-analysis → nucleotide-sequence / molecular-protocol`
ChronometricAge	`event`-scoped `chronometric-age` (+ protocol / determined-by agent); also reachable from occurrence via event
Identifier / Reference extensions	fan-out from `event|occurrence|material|survey-*-identifier` and `*-reference` / `protocol-reference` / `identification-reference` / … → `bibliographic-resource`

Table 9: Primary paths from Event/Occurrence cores — cardinality as in the Java DSL.

Agent **names** for `recordedBy` / `identifiedBy` / `eventConductedBy` / `georeferencedBy` are often resolved by **weak ID columns straight to `agent`** (`AgentMapping`), not only via `*-agent-role` tables. Role tables that **are** used today include `identification-agent-role`, `material-agent-role`, and `media-agent-role`.

7.9.6 Tables not in the current projection (24)

Present in the official catalogue but **not** referenced by the mapping configs (so they do not reach GBIF.org Event/Occurrence indexes unless some other path copies a scalar):

`agent-agent-role`, `agent-identifier`, `agent-media`, `chronometric-age-agent-role`, `chronometric-age-protocol`, `event-agent-role`, `geological-context-media`, `media-assertion`, `media-identifier`, `media-provenance`, `molecular-protocol-agent-role`, `occurrence-agent-role`, `occurrence-protocol`, `organism-assertion`, `organism-identifier`, `organism-interaction`, `organism-interaction-agent-role`, `organism-interaction-assertion`, `organism-interaction-media`, `organism-interaction-reference`, `organism-reference`, `organism-relationship`, `resource-relationship`, `survey-agent-role`.

A package that is mostly interactions / relationships can still be valid DwC-DP and can still be stored as Parquet on HDFS — but almost none of that graph reaches cross-dataset search. New biology ⇒ new hand-authored fragments, not automatic coverage.

> **Practical reading for OBIS**
>
> If you reimplement this in DuckDB, treat the **53 tables + `via` list + paths above** as the v1 fragment registry to copy — not the full DwC-DP catalogue. Anything outside that set is "store and download as tables" (Track B), not "index into occurrence/event search" (Track A), until you deliberately add fragments.

8 Implementing a GBIF-like projection with DuckDB

DuckDB is a good fit for OBIS-scale **per-dataset or batch** projection. You do **not** need HDFS or Spark to reproduce the mapping **logic**. What GBIF uses Spark for is cluster throughput over a global crawl; the recipes themselves are ordinary joins, filters, and aggregations.

Two different DuckDB jobs are easy to confuse:

Job	What DuckDB should do
Track B — multi-table subset export	What `query_subset.py` already does: filter occurrences → collect keys → export related Parquet/JSONL **tables without flattening**. Best default for downloads and analysis.
Track A — search projection	GBIF's `DwcDpVerbatimConverter` analogue: flatten a **declared** neighbourhood into occurrence/event documents (+ nested extension lists) for Elasticsearch. This section focuses here.

8.1 Recommended tool split (practical)

Tool	Role
DuckDB	All joins, cardinality checks, field merges, Parquet in/out. Run per dataset or over a dated lake partition. Primary engine for OBIS.
Python	Orchestration: read `datapackage.json`, prune the fragment registry, render SQL, write ES bulk JSON / GeoParquet, unit-test fragments. Keep SQL templates in files or a small YAML registry — do not bury 2 000-line queries in one script.
Elasticsearch	Track A serving layer only. Bulk-index the projected documents DuckDB produced. Do not ask ES to join DwC-DP tables.
Object storage / NAS	Dataset-scoped Parquet lake (`…/<dataset_id>/<table>.parquet`). Same layout Track B already uses.
Spark / Flink	**Optional later**, only if a single DuckDB process cannot finish nightly reindex of the full corpus. Port the **same** SQL/recipes; do not redesign the model just to use a cluster.
dbt (optional)	If the fragment registry grows large, dbt models map cleanly onto "one fragment = one model" with tests for `exactlyOne` violations.

You do **not** need a Java port of `DwcDpMappingEngine`. You **do** need an explicit fragment registry — otherwise DuckDB will tempt you into one giant `SELECT` that silently changes behaviour when a new table appears.

8.2 Shape of a fragment registry

Represent each GBIF-style fragment as data, then compile to SQL only for tables present in the dataset:

```yaml
# sketch — not production schema
name: occurrence_accepted_identification
kind: enrichment          # enrichment | extension
requires_tables: [occurrence, identification]
join:
  - to: identification
    on: identification.occurrence_fk = occurrence.occurrence_pk
    filter: "isAcceptedIdentification IS TRUE"
    cardinality: exactly_one
targets:
  scientificName: { prefer: [identification.scientificName], agg: first_non_null }
  identifiedBy:   { prefer: [occurrence.identifiedBy, identification.identifiedBy], agg:
first_non_null }
```

```yaml
name: occurrence_media
kind: extension
row_type: Multimedia
requires_tables: [occurrence, occurrence-media, media]
join:
  - to: occurrence-media
    on: occurrence-media.occurrence_fk = occurrence.occurrence_pk
    cardinality: fan_out
  - to: media
    on: media.media_pk = occurrence-media.media_fk
    cardinality: exactly_one
limit_per_parent: 50
targets:
  identifier: { prefer: [media.identifier], agg: first_non_null }
  # …
```

Pruning = skip any fragment whose `requires_tables` are missing from `datapackage.json` (or from the lake folder). That is the DuckDB equivalent of GBIF's `CompiledMappingDatasetPruner`.

8.3 SQL patterns that match the Java DSL

Identity (semantic ID, else structural PK, else URN):

```sql
COALESCE(
  nullif(o.occurrenceID, ''),
  nullif(o.occurrence_pk, ''),
  'urn:obis:dwcdp:occurrence:' || o.dataset_id || ':' || o.occurrence_pk
) AS document_id
```

`exactlyOne` enrichment (accepted identification) — reject multiples:

```sql
WITH accepted AS (
  SELECT i.*,
         count(*) OVER (PARTITION BY i.occurrence_fk) AS n
  FROM identification i
  WHERE i.isAcceptedIdentification IS TRUE
)
SELECT o.*,
       a.scientificName AS scientificName_from_id
FROM occurrence o
```

```
LEFT JOIN accepted a
  ON a.occurrence_fk = o.occurrence_pk
 AND a.n = 1;          -- 0 or >1 accepted ⇒ no enrichment
```

`fanOut` extension (occurrence media), with per-parent cap:

```
SELECT o.occurrence_pk AS coreid,
       m.*
FROM occurrence o
JOIN "occurrence-media" om ON om.occurrence_fk = o.occurrence_pk
JOIN media m ON m.media_pk = om.media_fk
QUALIFY row_number()
  OVER (PARTITION BY o.occurrence_pk ORDER BY m.media_pk) <= 50;
```

Assertions → eMoF-shaped extension rows (illustrative):

```
SELECT o.occurrence_pk AS coreid,
       ea.assertionType  AS measurementType,
       ea.assertionValue AS measurementValue,
       ea.assertionUnit  AS measurementUnit
FROM occurrence o
JOIN "occurrence-assertion" ea ON ea.occurrence_fk = o.occurrence_pk;
-- plus unioned SELECTs for material-/event-routed assertion fragments
```

Parent event walk for downloads or event-doc building (already in `query_subset.py`):

```
WITH RECURSIVE event_keys AS (
  SELECT dataset_id, interpreted.eventID AS event_id, 0 AS depth
  FROM filtered_occurrences
  WHERE interpreted.eventID IS NOT NULL
  UNION ALL
  SELECT e.dataset_id, e.interpreted.parentEventID, k.depth + 1
  FROM read_parquet('…/*/event.parquet') e
  JOIN event_keys k
    ON e.dataset_id = k.dataset_id
   AND e.interpreted.eventID = k.event_id
  WHERE e.interpreted.parentEventID IS NOT NULL AND k.depth < 10
)
SELECT DISTINCT dataset_id, event_id FROM event_keys;
```

Always scope joins with `dataset_id` (or run **one dataset at a time**). That is OBIS's substitute for GBIF reminting `_pk` inside a single crawl workspace.

8.4 Building Track A documents in DuckDB

A pragmatic batch for one dataset:

1. Load or `read_parquet` only tables listed in that dataset's package / folder.
2. Build a wide `occurrence_core` table by left-joining each **enrichment** fragment (organism, accepted identification, single evidence material, …) using the `exactlyOne` pattern.
3. Build extension tables (`occurrence_multimedia`, `occurrence_emof`, …) with `fanOut` queries; optionally `json_group_array` them onto the core row if Elasticsearch wants nested JSON.
4. Write `occurrence_projected.parquet` (and/or NDJSON for bulk indexing).
5. Repeat for `event_projected` when you maintain an event index.

Example nesting step:

```
COPY (
  SELECT c.*,
         coalesce(mm.media, []) AS multimedia,
         coalesce(mf.facts, []) AS measurements
  FROM occurrence_core c
  LEFT JOIN (
    SELECT coreid, json_group_array(json_object('identifier', identifier, …)) AS media
    FROM occurrence_multimedia GROUP BY coreid
  ) mm ON mm.coreid = c.document_id
  LEFT JOIN (
    SELECT coreid, json_group_array(json_object(
             'measurementType', measurementType,
             'measurementValue', measurementValue)) AS facts
    FROM occurrence_emof GROUP BY coreid
  ) mf ON mf.coreid = c.document_id
) TO 'occurrence_projected.parquet' (FORMAT PARQUET);
```

Keep event-level MoF **off** the occurrence document unless you consciously accept duplication — that is the whole point of splitting Track A and Track B.

8.5 What DuckDB should not try to be

- **A live multi-hop query API over raw DwC-DP for every user filter.** Serve Track A from Elasticsearch (or a projected Parquet + simple filters); use DuckDB offline or for downloads.
- **A substitute for Frictionless validation.** Keep `dwc-dp-analyser`-style checks (or frictionless-py) at ingest; projection assumes mostly valid packages.
- **An automatic "follow all foreignKeys" engine.** That recreates the flexibility problem GBIF deliberately avoided. New table support = new registry entry + tests.

8.6 Minimal implementation roadmap for OBIS

1. Freeze lake layout: `…/<dataset_id>/{occurrence,event,…}.parquet` + `datapackage.json` or a manifest of present tables.
2. Codify today's `query_subset.py` exports as Track B fragment registry (already close).
3. Add a Track A registry with 5–10 enrichments (event identity, organism, accepted identification, evidence material, occurrence DNA) and 2–3 extensions (occurrence MoF, occurrence media/DNA).
4. Emit `occurrence_projected.parquet` → bulk index; compare filters/results to the current API on the Branchiostoma / Crangonidae fixtures.
5. Grow fragments only when a product requirement appears (survey/Humboldt, material catalogue fields, …) — same governance model as GBIF's Java configs.

> **Takeaway**
>
> Publish-time DwC-DP is flexible; **index-time** DwC-DP is a deliberate subset. Cross-dataset query reuses an occurrence/event index fed by that subset. DuckDB can implement the same projection as Spark SQL recipes plus a pruneable fragment registry; Elasticsearch remains the serving layer; Spark is optional scale-out, not a conceptual requirement. Anything outside declared fragments stays invisible until someone adds a fragment — or until a native multi-table product ships.

9 One projection framework for DwC-A and DwC-DP?

The useful idea is narrower than "make DwC-A look like DwC-DP" or "collapse everything to a DwC-A star." It is this: **a projection framework only needs tables, join paths, cardinality rules, and a fixed target shape.** Both DwC-A and DwC-DP are already that, once parsed.

DwC-A: core table + extension tables linked by `id` / `coreid` (a star, but still a relational graph).
DwC-DP: many tables linked by `_fk` / `_pk` or weak ID fields (a richer graph).
Projection: declared fragments that walk those edges and emit a more fixed structure (search documents, eMoF rows, multi-table download slices, …).

So the **same** engine — pruneable fragment registry, `exactlyOne` / `fanOut`, merge policies, DuckDB SQL compilation — can run on either input. What differs is only how bytes become named tables, and which fragment paths are valid for a given dataset.

9.1 What should be identical

- Fragment registry format (requires tables, join `on`/`via`, cardinality, targets, enrichment vs extension).
- Execution semantics (prune if table absent; optional miss ≠ failure; caps like 50 media rows).
- Track A vs Track B operators (flatten to search docs vs export related table slices from a driving key set).
- Tests that assert "given these tables, projection emits …".

9.2 What should stay format-specific (thin)

- **Parsers / landers:** `meta.xml` + cores/extensions → tables; or `datapackage.json` + resources → tables.
- **Default join column names:** DwC-A uses `coreid` ↔ core `id`; DwC-DP uses `event_fk` ↔ `event_pk` (or weak `evidenceForOccurrenceID` ↔ `occurrenceID`). Fragments name the columns explicitly — the engine does not infer "the" FK style.
- **Which fragments exist:** DwC-A activates event/occurrence/MoF/DNA/multimedia paths; DwC-DP may additionally activate material/survey/assertion paths when those tables are present. Absent tables prune away for both.

9.3 What not to do

- Do not require DwC-A ingest to mint full DwC-DP `_pk` graphs or empty Agent/Protocol tables just to "use the framework."
- Do not make the framework auto-discover joins from `meta.xml` or `datapackage.json` foreignKeys — that reintroduces unbounded flexibility. Fragments stay declared.
- Do not equate "same projection framework" with "same target must be DwC-A star only." The fixed target can be OBIS lake tables, ES documents, or both; the framework is the walker/emitter, not the philosophy of the target.

9.4 Verdict

Yes: treat projection as format-agnostic over **tables + relationships → fixed structure**. DwC-A and DwC-DP then share one framework by construction. Keep two small landers at the edge; avoid two parallel projection codebases. That is the best option for OBIS among realistic designs — and it matches what your DwC-A separate-tables prototype is already gesturing at, with DwC-DP as more tables in the same model rather than a second stack.

> **Recommendation**

Share one projection framework for DwC-A and DwC-DP. Inputs are just tables with relationships; outputs are a declared fixed structure. Format-specific work stops at landing rows into those tables and at which fragments a dataset can activate.

10 How querying and downloading likely work in practice

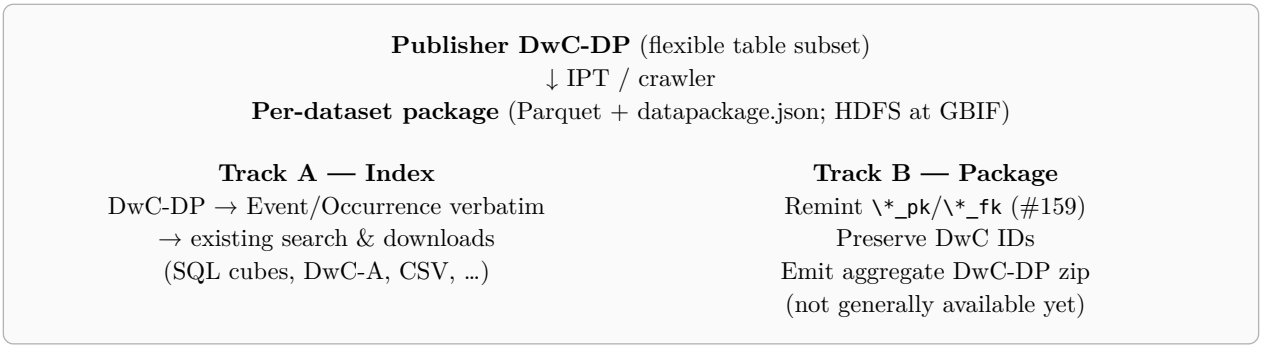


Figure 5: Two tracks: occurrence/event query now; relational DwC-DP aggregate export when ready.

10.1 Cross-dataset query (near term)

From a user's point of view on GBIF.org, little changes at first: you still search occurrences (and, where enabled, events), apply taxonomic/geographic/temporal filters, and download CSV, DwC-A, or SQL/cube results. Under the hood, some of those records may have originated as DwC-DP and been flattened by the mapping engine; others still come from classic DwC-A or from Camtrap DP → DwC-A.

What you **cannot** yet do as a first-class product is: "give me every DwC-DP table connected to these occurrences across datasets," or "filter on an organism-interaction predicate that never projected into the occurrence index." Rich DwC-DP-only structures are only as queryable as the mapping fragments copied them into core fields or known extensions (eMoF, DNA, Humboldt, …).

10.2 Cross-dataset download (two possible products)

It helps to separate three download shapes that often get conflated:

Product	Status / mechanism	Role of #159
Classic / SQL / cube downloads	Production today; fed by interpreted occurrence/event records	Indirect — DwC-DP must map into those records first
Aggregate DwC-DP download	Design-enabled by #159; product not generally shipped	Direct — remint structural keys, preserve semantic IDs
Source-dataset DwC-DP	Publisher archives / IPT / dataset page	Not required (single-dataset uniqueness)

Manon Gros's comment on #159 remains the pragmatic stance for Track A: interpreted network downloads may carry GBIF-oriented keys and flattened fields, while users who need pristine publisher packages download at the source. Track B is what #159 makes **honest** when GBIF wants to hand users a multi-dataset DwC-DP that still joins correctly **and** still shows original `occurrenceID` values. Until Track B ships, "cross-dataset DwC-DP download" is a standards capability, not a button on GBIF.org.

11 What OBIS could do without HDFS

OBIS faces the same tension GBIF does — event-level measurements duplicated onto occurrences, growing DNA payloads, and pressure to support richer event/survey identifiers (BBNJ batch IDs, WIGOS/platform codes) — but its infrastructure is different: GeoParquet and Elasticsearch today, with a prototype of **separate Parquet tables per dataset** queried by DuckDB (iobis/pipeline `separate-tables`, local `scripts/query_subset.py`). None of that requires HDFS or Spark. The useful thing to borrow from GBIF is the **discipline** (bounded projection for search; multi-table export for downloads), not the cluster.

11.1 Why not flatten everything in OBIS either?

The current OBIS API often returns event-level MoF nested under occurrences (inheritance for convenience). That made R-package workflows easy, but it multiplies storage and response size when a station has dozens of sediment parameters and many taxa. DwC-DP will make the same problem worse if every related assertion is copied onto every occurrence. GBIF's answer is: keep search documents relatively flat/known-shaped; let downloads carry related tables. OBIS can do the same split without waiting for a DwC-DP zip format.

11.2 Recommended split (mirror GBIF's two tracks)

Track	OBIS-shaped implementation
A — Search / API	Keep an occurrence (and optionally event) Elasticsearch index for filters and maps. Nest **directly linked** extensions only (e.g. occurrence DNA), not bulk event sediment MoF. Denormalise filter fields (coords, depth, BBNJ/WIGOS IDs, dates) onto the search document even if they originated on a parent event.
B — Download / analysis	Export **subsets of separate tables** — occurrence, event, event/occurrence MoF, DNA, and later DwC-DP tables — keyed by `(dataset_id, …ID)`. Client joins in DuckDB or R. Do **not** flatten event MoF onto every occurrence in the download.

Physical store: object storage or a NAS of Parquet files is enough. DuckDB already spans datasets with globs such as `read_parquet('…/*/occurrence.parquet')`. Dataset folders in the prototype look like `temp_parquet_tables/<dataset_uuid>/{occurrence,event,…}.parquet` plus `eml.xml`.

11.3 Keys for the separate-tables pipeline: composite now, structural when packaging

The open design question for `separate-tables` is whether OBIS should already mint GBIF-style technical keys (`event_pk`, `occurrence_pk`, `event_fk`, …) in every Parquet export, or rely on **composite** joins `(dataset_id, Darwin Core ID)` for lake queries and multi-table downloads.

> **Recommendation**
>
> **Do not block** the multi-table export on reminted `_pk` / `_fk` columns. Standardise composite joins everywhere in the lake and in download docs: always `dataset_id` + semantic ID (never a DwC ID alone). Add structural keys when DwC-A / DwC-DP **package** downloads (or opaque API handles) are near — mint them **alongside** composites, with GBIF naming, and never overwrite publisher ID fields.

That follows directly from the #159 analysis earlier in this briefing (Section 4.4): lake analytics can use composite keys; a multi-dataset Frictionless zip still needs remintable structural keys so semantic IDs stay untouched.

11.3.1 Three layers (do not collapse them)

Layer	Join identity	Fits composite downloads?
Parquet lake / DuckDB subsets	`(dataset_id, eventID)` / `(dataset_id, occurrenceID)` (+ parent walk)	Yes — this **is** the intended Track B model
Multi-dataset Frictionless DwC-DP zip	Remintable `event_pk` / `occurrence_pk` + `_fk` (#159)	No — composite on semantic IDs is not a Frictionless substitute
Classic DwC-A aggregate download	Structural `id` / `coreid` (GBIF: `gbifID`)	Same problem as #159, smaller star schema

Table 10: Where each key style belongs in the OBIS design.

The current prototype already implements the lake row of that table (`scripts/query_subset.py`: filter occurrences → extract `(dataset_id, occurrenceID)` → walk `(dataset_id, eventID)` / `parentEventID` → join extension tables). Cross-dataset DwC-A downloads are the README item that still "will require new structural keys"; that is a **packager** concern, not a reason to remint every lake file today.

11.3.2 When technical keys do make sense early

Add (or plan to mint) `_pk` / `_fk` in the pipeline if any of these are within one or two releases:

1. Cross-dataset **DwC-A** downloads that need a mintable `id` / `coreid` without rewriting `occurrenceID`.
2. Multi-dataset **DwC-DP** package downloads that must Frictionless-validate.
3. Missing, blank, or colliding `eventID` / `occurrenceID` **inside** a dataset (composite on semantic IDs then fails).
4. Opaque row handles for Elasticsearch / API nesting (`obisID`-like), independent of publisher IDs.

Otherwise wait: premature reminting in every Parquet file is easy to get wrong and awkward to migrate.

11.3.3 Staying close to GBIF without copying Spark

GBIF's post-#159 pattern (and the projection code that prefers semantic ID then falls back to structural PK):

- **Semantic:** keep `eventID` / `occurrenceID` / … untouched (provenance).
- **Structural:** package joins on `event_pk`, `occurrence_pk`, `event_fk`, `parentEvent_fk`, …
- **Lake stamp:** often also `datasetKey` / `dataset_id` on rows — complementary, not alternative.

OBIS-shaped implementation:

A. Lake (now) — composite, as intended for downloads

```
occurrence:  dataset_id + interpreted.occurrenceID   (+ eventID)
event:       dataset_id + interpreted.eventID        (+ parentEventID)
extensions:  dataset_id + occurrenceID | eventID
```

Downloads = filtered subsets of those tables; clients join on the same composites. No reminting.

B. Optional thin structural layer (forward-compat)

Per dataset crawl, mint opaque keys **in addition to** `dataset_id` and DwC IDs — do not replace them:

Column	Role
`event_pk` / `occurrence_pk`	Unique within the dataset (ULID, sequential, or deterministic hash)
`event_fk` on occurrence / event-MoF	→ `event.event_pk`
`parentEvent_fk`	→ parent `event_pk`
`occurrence_fk` on occurrence-MoF / DNA	→ `occurrence_pk`

Table 11: Optional structural columns in the lake, GBIF naming.

Mint rule aligned with GBIF: aggregators may remint structural keys; never overwrite DwC ID fields. A deterministic option that survives reprocessing:

`occurrence_pk = uuid5(dataset_id, "occurrence:" + occurrenceID)`
(and refuse / fall back to a row hash if `occurrenceID` is null).

Lake queries can keep using composites; packaging later uses `_pk` / `_fk`.

C. At download assembly time (when packages are needed)

- **Multi-table Parquet/CSV zip (Track B):** stamp `dataset_id` on every row; document joins as `(dataset_id, …)`. No remint required.
- **Frictionless DwC-DP aggregate:** remint `_pk` / `_fk` globally unique across datasets in the zip (e.g. UUID); leave `eventID` / `occurrenceID` as published; optionally keep `datasetKey` for lake round-trips.
- **DwC-A aggregate:** put reminted values in `id` / `coreid`; map DwC IDs as ordinary terms (same story as `gbifID`).

11.3.4 Practical next step for `separate-tables`

1. Ship and document composite joins as the lake / download contract.
2. Optionally add nullable `event_pk` / `occurrence_pk` / FKs with deterministic minting **now**, without changing DuckDB join paths.
3. Wire a packager to `_pk` / `_fk` only when DwC-A or DwC-DP aggregate export lands — reminting is a serialization job, not a lake rewrite.

11.4 A small "mapping plan" without Java

The fragment-registry approach is spelled out with SQL in the chapter **Implementing a GBIF-like projection with DuckDB**. Track B (multi-table export) is what `query_subset.py` already approximates; Track A (search projection) is the additional flatten step. Conceptually both share the same pruneable recipes:

```
filtered_occurrences
  → keys (dataset_id, occurrence_id)
  → recursive event keys via parentEventID
  → enrichment joins (exactly_one) and/or extension fan-outs
  → either wide search documents (Track A) or separate table slices (Track B)
```

Driver already used for Track B fixtures:

```
python scripts/query_subset.py \
  --data-root /path/to/temp_parquet_tables \
  --genus Branchiostoma \
  --output ./output/branchiostoma \
  --format jsonl
```

On the Branchiostoma test datasets, Track B returned matching occurrences, their events (including parents), event-level MoF counted once per event, and occurrence-level MoF/DNA without multiplying sediment rows onto every taxon record.

11.5 Extending the plan toward DwC-DP

Treat each DwC-DP table OBIS cares about as another optional export recipe, with the same prune-if-absent behaviour as GBIF's dataset pruner: if `material.parquet` is missing for a dataset, skip that recipe.

DwC-DP table (examples)	Join recipe (scoped by dataset_id)
`occurrence`	Driving set from taxonomy / geography / DNA filters
`event`	Via occurrence→event, then walk `parentEventID` recursively
`event-assertion` / MoF	Via event keys; ship as its own file (eMoF-shaped or native assertion)
`nucleotide-analysis` + sequence	Via occurrence or material evidence path
`survey` / `survey-target`	Via event; either separate tables or Humboldt-like columns on search docs
`organism-interaction`	Defer until there is an API/download use case (GBIF also omits this today)

A practical rollout order for OBIS: (1) stabilise separate Event/Occurrence/MoF/DNA Parquet + DuckDB downloads; (2) slim ES nesting; (3) add material/survey/agent only when a product needs them; (4) optionally emit DwC-DP zips with reminted `_pk`/`_fk` for interchange. That is the same discipline as GBIF's limited fragment set — implemented as SQL/Python instead of Spark `MappingPlan`s.

11.6 Elasticsearch / API implications

- **Search documents:** nest only extensions users filter on directly (DNA is the obvious case). Drop mass event-MoF nesting from the default occurrence document.
- **Downloads:** return a folder or zip of table slices; document that event-level measurements live in `event_measurementorfact` (or `event-assertion`), not under each occurrence.
- **R package:** keep "quick occurrence + direct extensions" via the API; point users at downloads when they need full event context.
- **Policy filters:** denormalise BBNJ batch IDs, WIGOS/platform codes, and similar onto the occurrence or event search document so Track A filters stay fast even when Track B stays multi-table.

11.7 Compatibility with GBIF

The multi-table lake does not need to look like GBIF's `verbatim.avro`. Interoperability matters at the **exchange** boundary: publish DwC-A or DwC-DP that GBIF's `EventDwcaMapping` / `OccurrenceDwcaMapping` understand, **or** run an OBIS-side flatten into Event/Occurrence + eMoF/DNA before pushing upstream. Trying to make Elasticsearch store the full DwC-DP graph "like GBIF's future lakehouse" is optional and can wait; borrowing the two-track split is the near-term win.

12 Open questions (still unresolved publicly)

These remain open in public sources; closed issue #159 does not answer them.

1. **When** will GBIF.org offer something like `format=DWCDP` for aggregate downloads — and will it cover mixed DwC-A + DwC-DP result sets from day one?

2. **Which closure** of related tables is included for a given occurrence predicate (parents only? all agents? all protocols?)?
3. Will structural keys in those downloads be opaque GBIF IDs, `datasetKey:` prefixes, resolvable API identifiers, or freshly minted per download?
4. How will tables that exist only in DwC-DP (Survey Target, Material derivation, …) be populated for rows that originally arrived as DwC-A?
5. Will ClickHouse / Iceberg work in `datapackages-pipeline` become a user-facing SQL surface over native DwC-DP tables, or remain an internal analytics path?

Until those product choices land, the safe reading of public evidence is: **search stays projected; full-graph DwC-DP is for publishing and (later) specialised export.**

13 Conclusions

1. **Issue #159 is the decisive modelling fix** for aggregate DwC-DP: structural keys may be reminted; Darwin Core identifiers must be preserved. Adding `dataset_id` to lake joins is complementary and useful — it does **not** replace `_pk`/`_fk` inside a multi-dataset Frictionless zip.
2. **That fix is necessary but not sufficient** for cross-dataset query: flexibility of optional tables and relationships remains a separate challenge. Projection uses a **governing plan + fragments** executed as Spark table joins (enrichment vs fan-out), not a full-graph or KV-store indexer.
3. **GBIF's visible engineering bet** is a curated DwC-DP → Event/Occurrence projection in `gbif/pipelines` (1365 / `DwcDpVerbatimConverter` + `EventDwcaMapping` / `OccurrenceDwcaMapping`), plus native package tooling (Parquet, ES, ClickHouse) in parallel — not a generic FK walker.
4. **Native multi-dataset DwC-DP download** is now schema-feasible and widely agreed as desirable; it is not yet a documented, generally available GBIF product.
5. **OBIS can follow the same two-track idea** with DuckDB + dataset-scoped Parquet: composite `(dataset_id, …ID)` keys for the lake and multi-table downloads; remint `_pk`/`_fk` only when emitting DwC-A / DwC-DP packages (optionally mint structural columns in the lake **alongside** composites for forward compatibility — see Section 11.3). Use a pruneable fragment registry for both multi-table downloads and — separately — search-document projection; Elasticsearch remains the API serving layer; Spark is optional scale-out only.
6. **For OBIS, one projection framework can cover DwC-A and DwC-DP:** both are tables with relationships projected into a fixed structure. Share the fragment engine; keep format-specific work at landing (and at which fragments a dataset activates). Do not require DwC-A to become full DwC-DP, and do not treat "same framework" as "target must be DwC-A star only."
7. **Expect a long hybrid period:** DwC-A remains the bulk of the network; Camtrap DP and DwC-DP enrich publishing; users query a projected index unless they download multi-table subsets or a future aggregate DwC-DP.

14 References

1. gbif/dwc-dp#159 — Primary and foreign keys in aggregate DwC-DPs (closed Jun 2026; schemas updated in commit `7c923f8`).
2. gbif/dwc-dp#150 — Weak foreign keys.
3. gbif/dwc-dp#139 — Plan to move repo / production artifacts.
4. TDWG — Darwin Core additions ratified (23 Jun 2026 news; ratification 26 May 2026).
5. DwC-DP Implementation Experience and Feature Report.
6. Darwin Core Data Package Guide (ratified).
7. DwC-DP Quick Reference Guide (development rendering; shows `_pk` / `_fk` and aggregation notes).
8. gbif/pipelines#1365 — Ingest DwC-DP to HDFS as Parquet and PR #1396.
9. `DwcDpVerbatimConverter.java` — routing to Event/Occurrence plans.
10. `spark/dwcdp/mapping/` — declarative DwC-DP → DwC-A framework (`EventDwcaMapping`, `OccurrenceDwcaMapping`, `AssertionMapping`, …).
11. gbif/datapackages-pipeline — DataPackage → Elasticsearch / ClickHouse.
12. gbif/dwc-dp-analyser — Frictionless + DuckDB validation.
13. gbif/ipt#1736 — Expand the data model and related IPT validation threads.
14. gbif/camtrap-dp-pipeline — Camtrap DP → DwC-A precedent.
15. GBIF Nodes Implementation Plan 2026.
16. GBIF training — Emerging publishing model: DwC-DP.
17. iobis/pipeline `separate-tables` — OBIS multi-table Parquet prototype.
18. Local OBIS subset exporter: `scripts/query_subset.py` (DuckDB over `dataset_id` + DwC IDs).

This briefing synthesizes public GitHub issues, repository code, and TDWG/GBIF documentation available as of early October 2026 (revised 5 Oct 2026 for OBIS separate-tables key strategy: composite lake joins vs #159 structural keys at package time). Internal GBIF roadmaps not published online may differ. Typographic report generated with Typst.